CONNECTING THE DOTS

ISC
High Performance

EAR ISC2025 tutorial: Advanced use cases and optimization

Julita Corbalan (julita.corbalan@eas4dc.com, julita.corbalan@bsc.es)
Benjamin Czaja (benjamin.czaja@surf.nl)
Martijn Kruiter (martijn.kruiten@surf.nl)
J

Advance use cases and optimization

- Codes in /projects/0/energy-course and GIT
- GIT: https://github.com/sara-nl/ISC-EAR-tutorial
- Get the examples and test them
  - https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/tutorials/monitoring_ear
  - GROMACS singularity
  - PyTorch
  - Palabos
  - PyTorch + dcgmi metrics
  - PyTorch + opt
- Data visualization
  - https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/tutorials/visualization
  - ear-job-visualizer tool

Energy optimization

Static and dynamic energy optimization with EAR

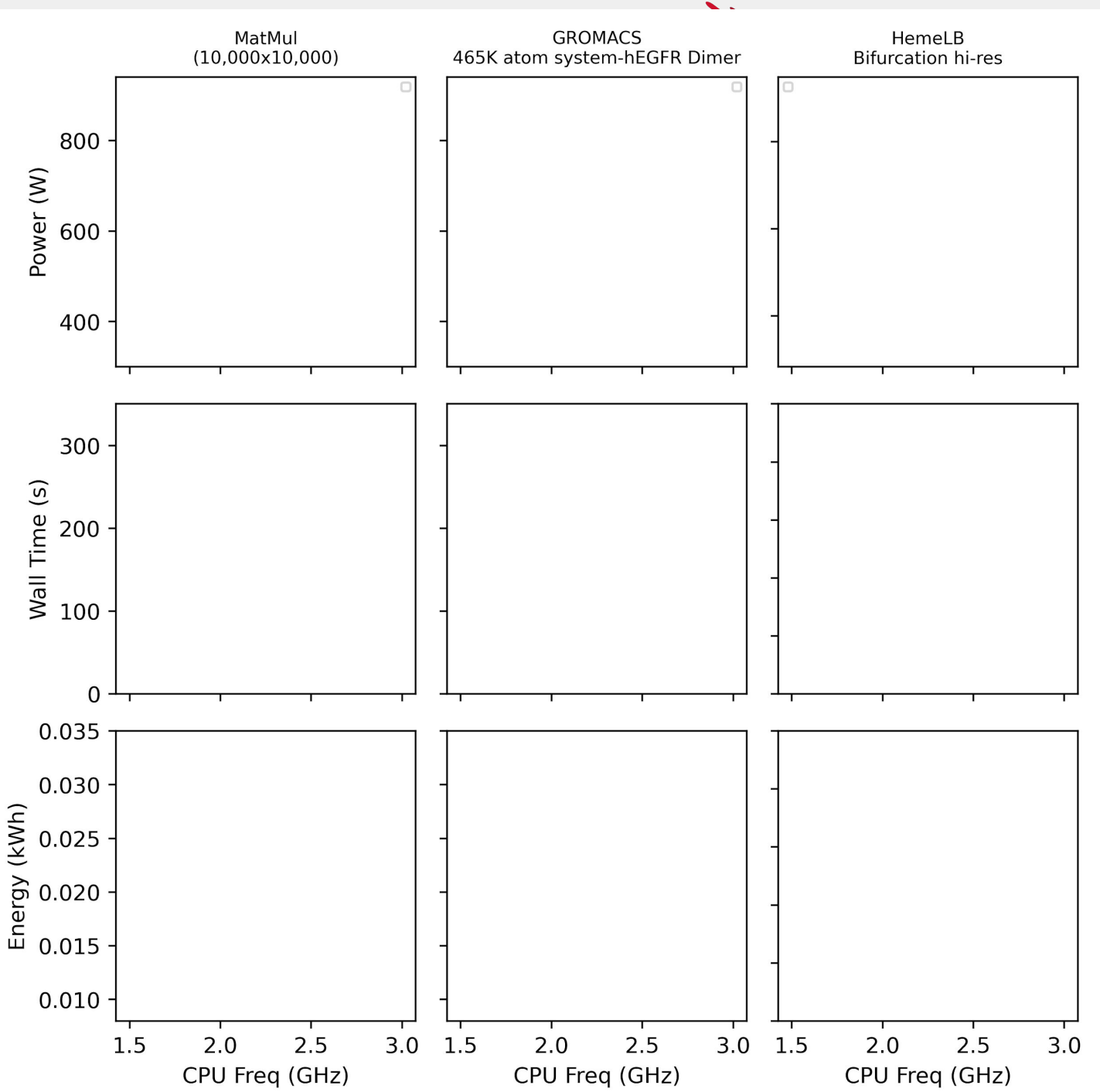
- Static optimization guided by performance optimization user expertise
- + Static optimization with manual CPU/GPU memory selection
  - THIS IS HARD TO REUSE AS PREVIOUSLY STATE

- + Dynamic optimization with energy policies

Dynamic Voltage Frequency Scaling on Snellius:

To boost, or not too boost, that is the question

- Rome “Zen2” Released 07 August 7 2019
  - Nominal Freq (2.6 Ghz)
- Genoa “Zen4” Released 10 November 2022
  - Nominal Freq (2.4 Ghz)



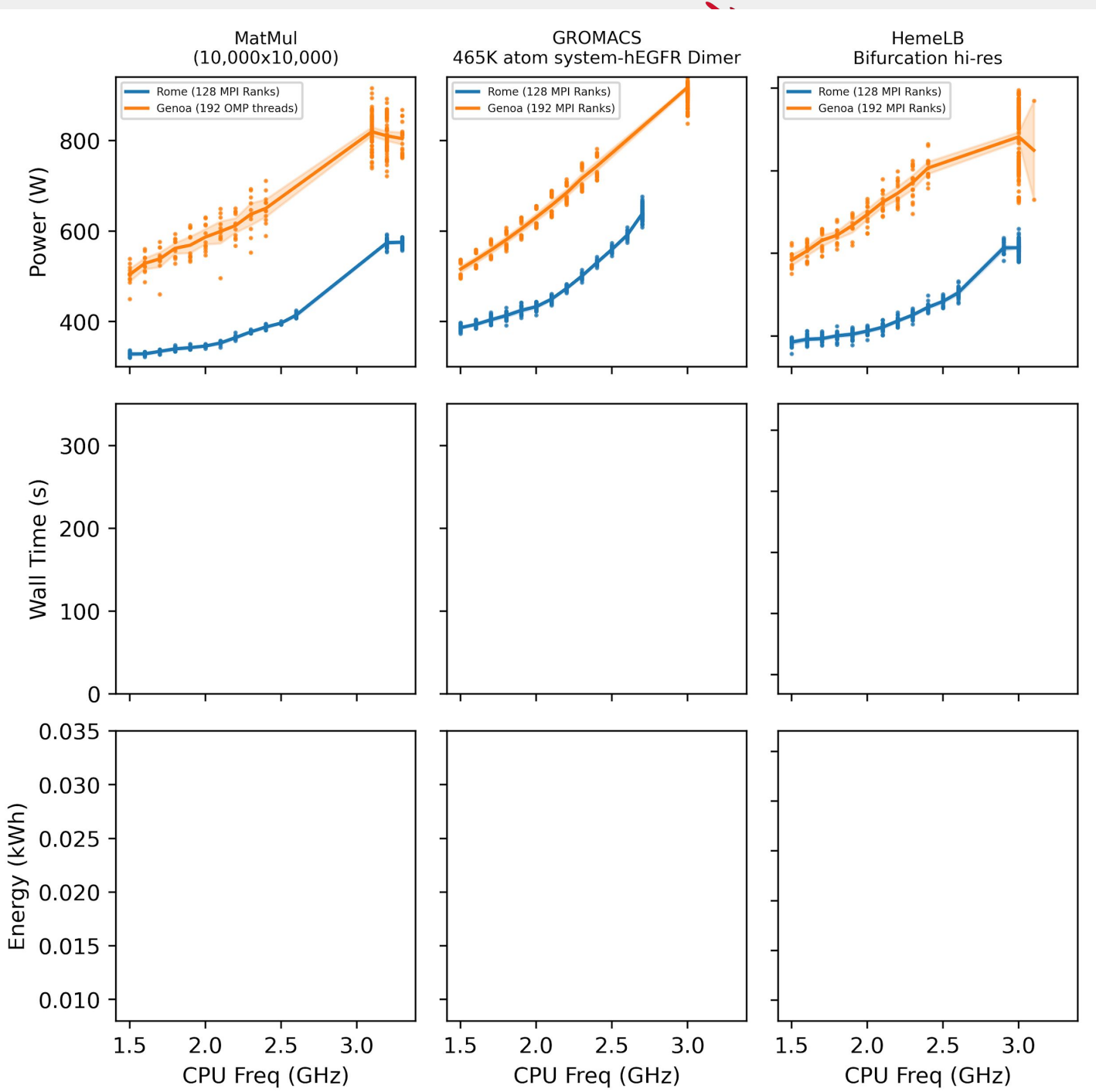
Dynamic Voltage Frequency Scaling on Snellius:

To boost, or not too boost, that is the question

- Rome “Zen2” Released 07 August 7 2019
  - Nominal Freq (2.6 Ghz)
- Genoa “Zen4” Released 10 November 2022
  - Nominal Freq (2.4 Ghz)

Observations:

1. Genoa draws more power
2. Each application has a different “power draw signature”



Dynamic Voltage Frequency Scaling on Snellius:

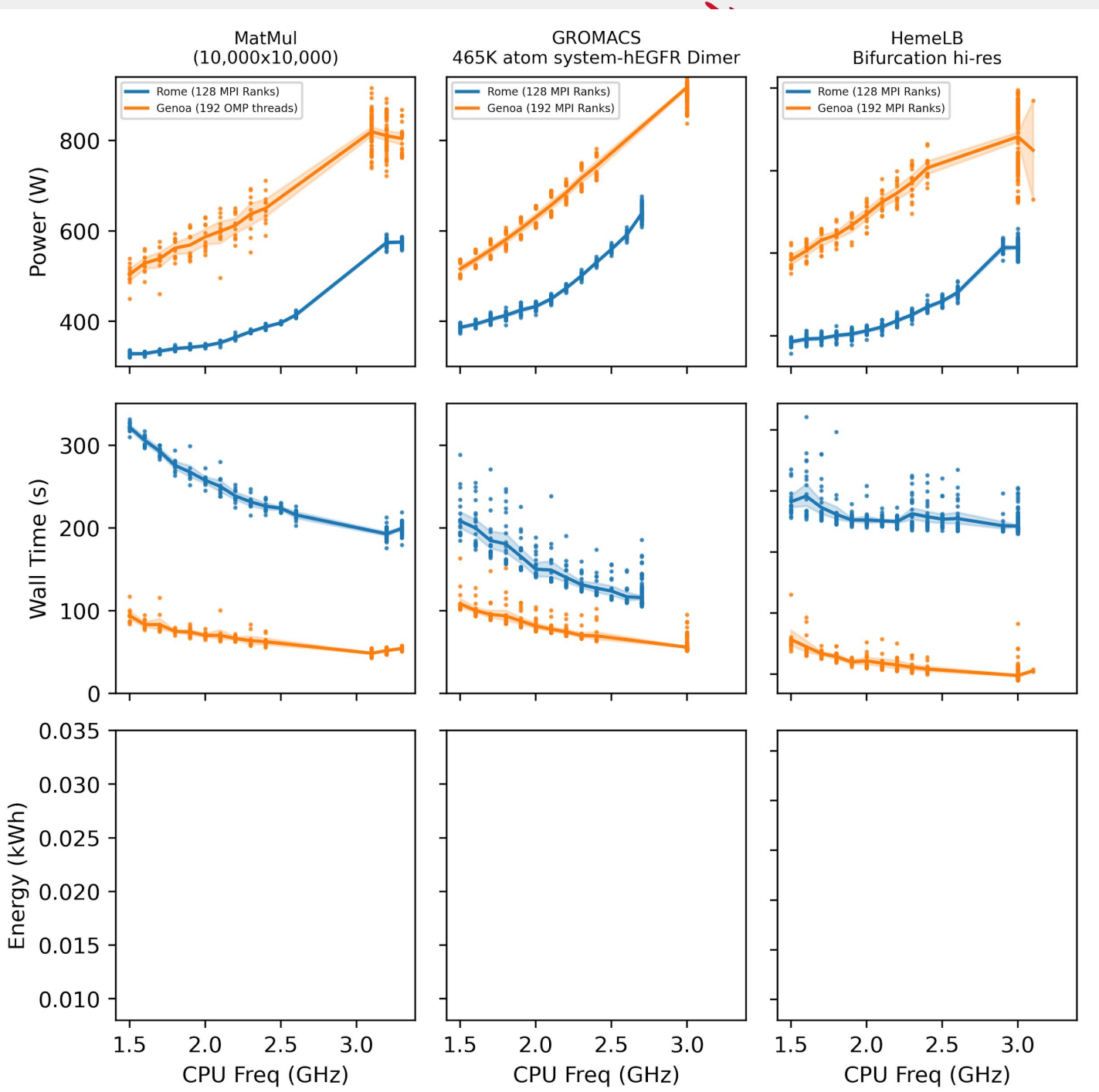
To boost, or not too boost, that is the question

- Rome “Zen2” Released 07 August 7 2019
  - Nominal Freq (2.6 Ghz)
- Genoa “Zen4” Released 10 November 2022
  - Nominal Freq (2.4 Ghz)

Observations:

1. Genoa is more performant
   - More Logical cores to devote to problem

2. In the 3 cases, Boost does not have the similar effect in performance vs power draw. i.e. its performance increase is flat as compared to its power draw



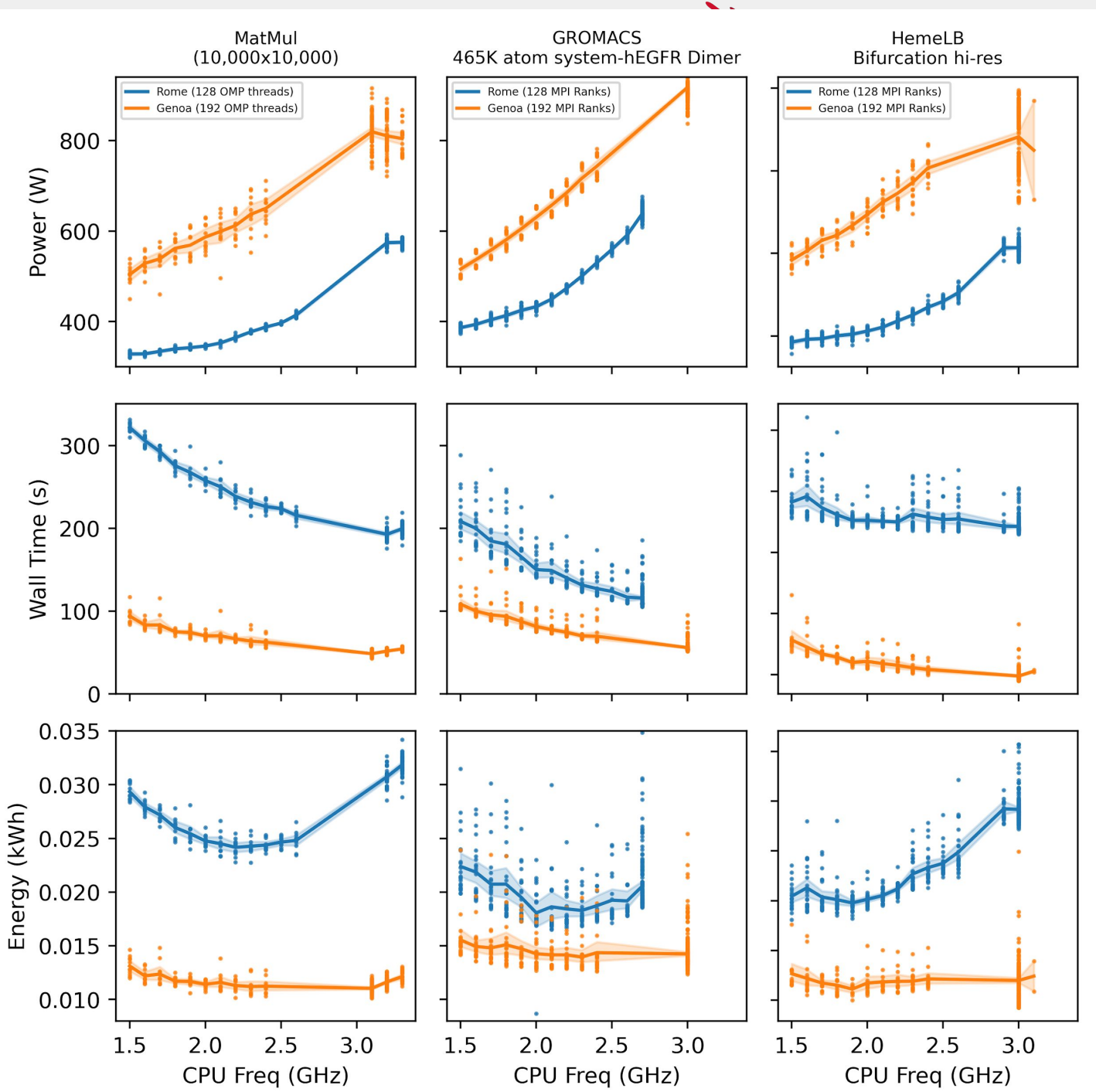
Dynamic Voltage Frequency Scaling on Snellius:

To boost, or not too boost, that is the question

- Rome “Zen2” Released 07 August 7 2019
  - Nominal Freq (2.6 Ghz)
- Genoa “Zen4” Released 10 November 2022
  - Nominal Freq (2.4 Ghz)

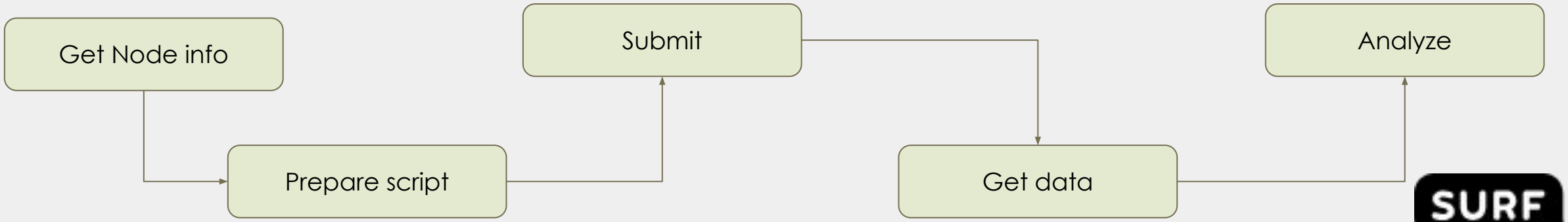
Observations:

1. Genoa is more energy efficient.
2. Each application has its own energy minimum.
3. **DVFS “by hand” is HARD!!!!!!!**



Static energy optimization with manual CPU/GPU frequency selection

- Optimal CPU/Memory/GPU frequency depends on the application, input data, architecture, number of nodes etc etc
- However, dynamic optimization is complementary to static resource optimization
- ????
- EAR offers in some architectures more CPU frequencies than available from the OS
- The enode_info command reports the EAR technical specification for the computational node



Get node info

```
$EAR_INSTALL_PATH/bin/tools/enode_info --cpu
EAR CPU info in node tcn2
EAR CPU info Topology: cpu_count        : 128
core_count       : 128
socket_count     : 2
….. // CPU details
EAR CPU info load
EAR CPU info API: EARD
EAR CPU info num devices:128
EAR CPU info list of CPU frequencies
PS0: id0, 2600000 KHz
PS1: id1, 2500000 KHz
PS2: id2, 2400000 KHz
PS3: id3, 2300000 KHz
PS4: id4, 2200000 KHz
PS5: id5, 2100000 KHz
PS6: id6, 2000000 KHz
PS7: id7, 1900000 KHz
PS8: id8, 1800000 KHz
PS9: id9, 1700000 KHz
PS10: id10, 1600000 KHz
PS11: id11, 1500000 KHz
EAR CPU info pstate nominal is 0, CPU freq = 2600000 KHz
EAR CPU info governor CPU[0] = conservative
….
EAR CPU info governor CPU[127] = conservative
EAR CPU info curr CPUF[0] = 2601000
..
EAR CPU info curr CPUF[127] = 2601000
```

```
#!/bin/bash
#SBATCH --job-name=sp
#SBATCH --ntasks=128
#SBATCH --ear=on
module purge
module load 2022
module load iimpi/2022a

export OMP_NUM_THREADS=1
export EARL_REPORT_LOOPS=1
srun --ear-policy=monitoring --ear-cpufreq=2500000 ./sp-mz.D.128
srun --ear-policy=monitoring --ear-cpufreq=2400000 ./sp-mz.D.128
srun --ear-policy=monitoring --ear-cpufreq=2300000 ./sp-mz.D.128
srun --ear-policy=monitoring --ear-cpufreq=2200000 ./sp-mz.D.128
```

Energy policies: Computational phases

- Monitoring:
  - Application analysis
  - Static energy optimization (Manual CPU/Memory/GPU freq selection)

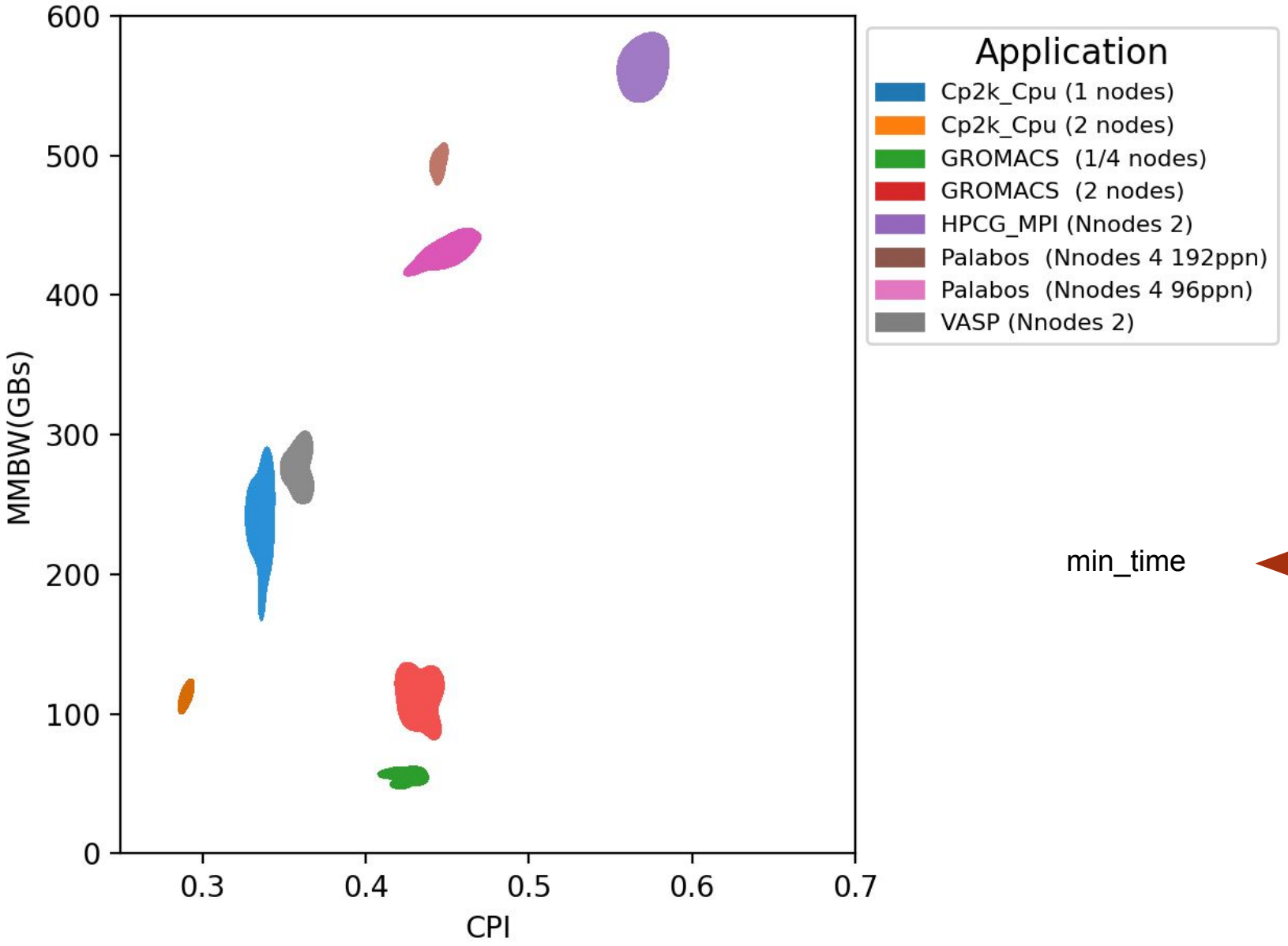
- Minimize energy to solution (min_energy): CPU optimization
  - EAR **reduces CPU frequency** to save energy with a maximum time penalty
  - Applications start at default CPU frequency and CPU frequency is (potentially) reduced
  - default frequency = nominal frequency
  - Memory frequency selected with a linear search
- Minimize energy to solution (min_energy) : GPU optimization
  - EAR **reduces GPU frequency** to save energy with a maximum time penalty
  - Applications start at default GPU frequency and GPU frequency is (potentially) reduced
  - default GPU frequency = max frequency
  - CPU frequency = max frequency (not modified)

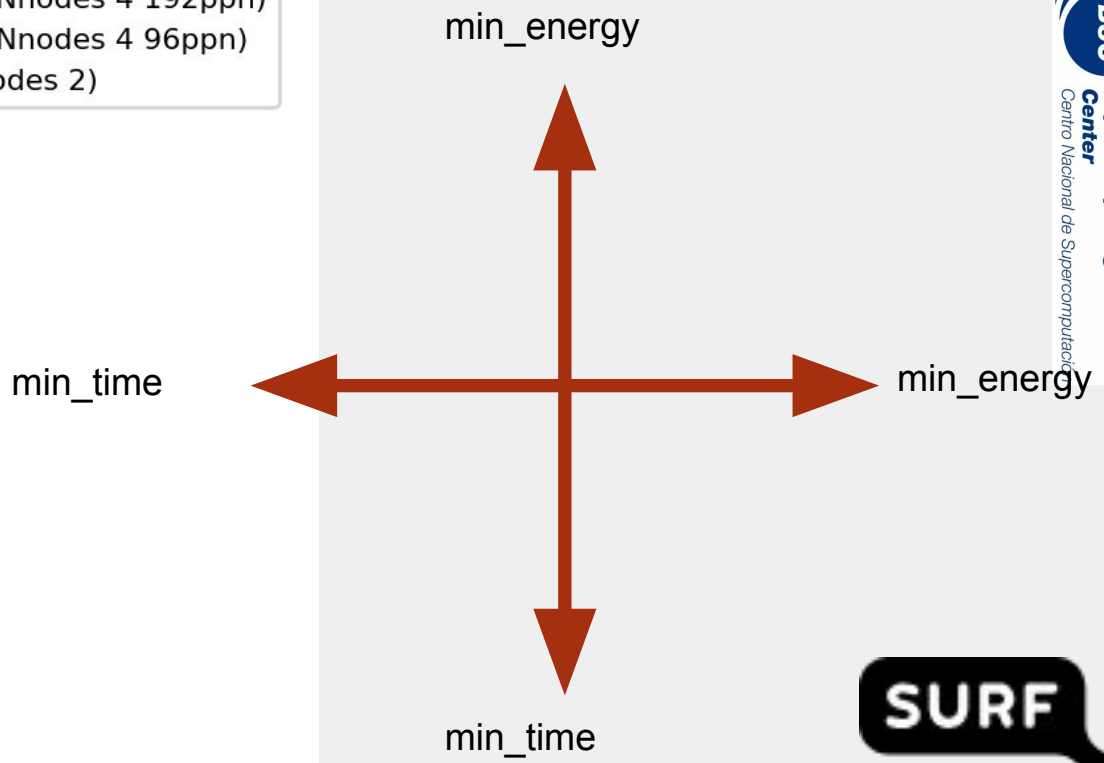
sbatch --ntasks=192 --partition=genoa sp.D.sh
sbatch --ntasks=192 --partition=genoa **–ear-policy=min_energy** sp.D.sh

KERNEL SP-MZ.D : ROME (Min_energy vs Nominal): Must be the same node for comparison

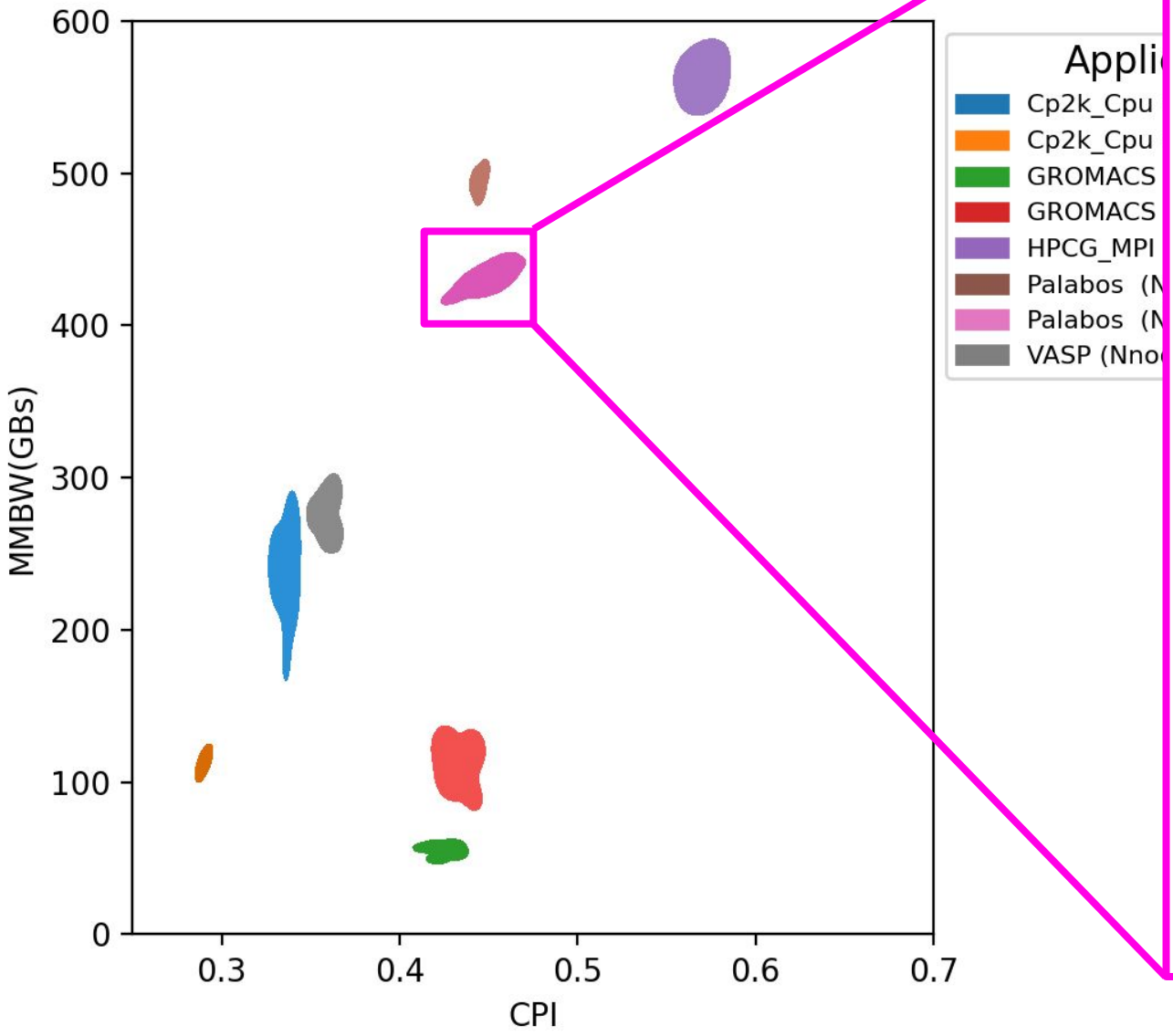
```
[julitac@int5]$ eacct -j 4896747.0
   JOB-STEP USER      APPLICATION      POLICY NODES AVG/DEF/IMC(GHz) TIME(s)    POWER(W) GBS     CPI   ENERGY(J)    GFLOPS/W IO(MBs) MPI%
G-POW (T/U)    G-FREQ  G-UTIL(G/MEM)
4896747-0    julitac    sp             ME     1    2.16/2.60/1.47   336.75    457.62    197.79  0.69  154102       2.9608   0.0     12.0  0.00/---       ---
---
[julitac@int5]$ eacct -j 4896738.0
   JOB-STEP USER      APPLICATION      POLICY NODES AVG/DEF/IMC(GHz) TIME(s)    POWER(W) GBS     CPI   ENERGY(J)    GFLOPS/W IO(MBs) MPI%
G-POW (T/U)    G-FREQ  G-UTIL(G/MEM)
4896738-0    julitac    sp             MO     1    2.57/2.60/1.47   329.67    519.01    202.73  0.79  171103       2.6666   0.0     11.0  0.00/---       ---
---
```

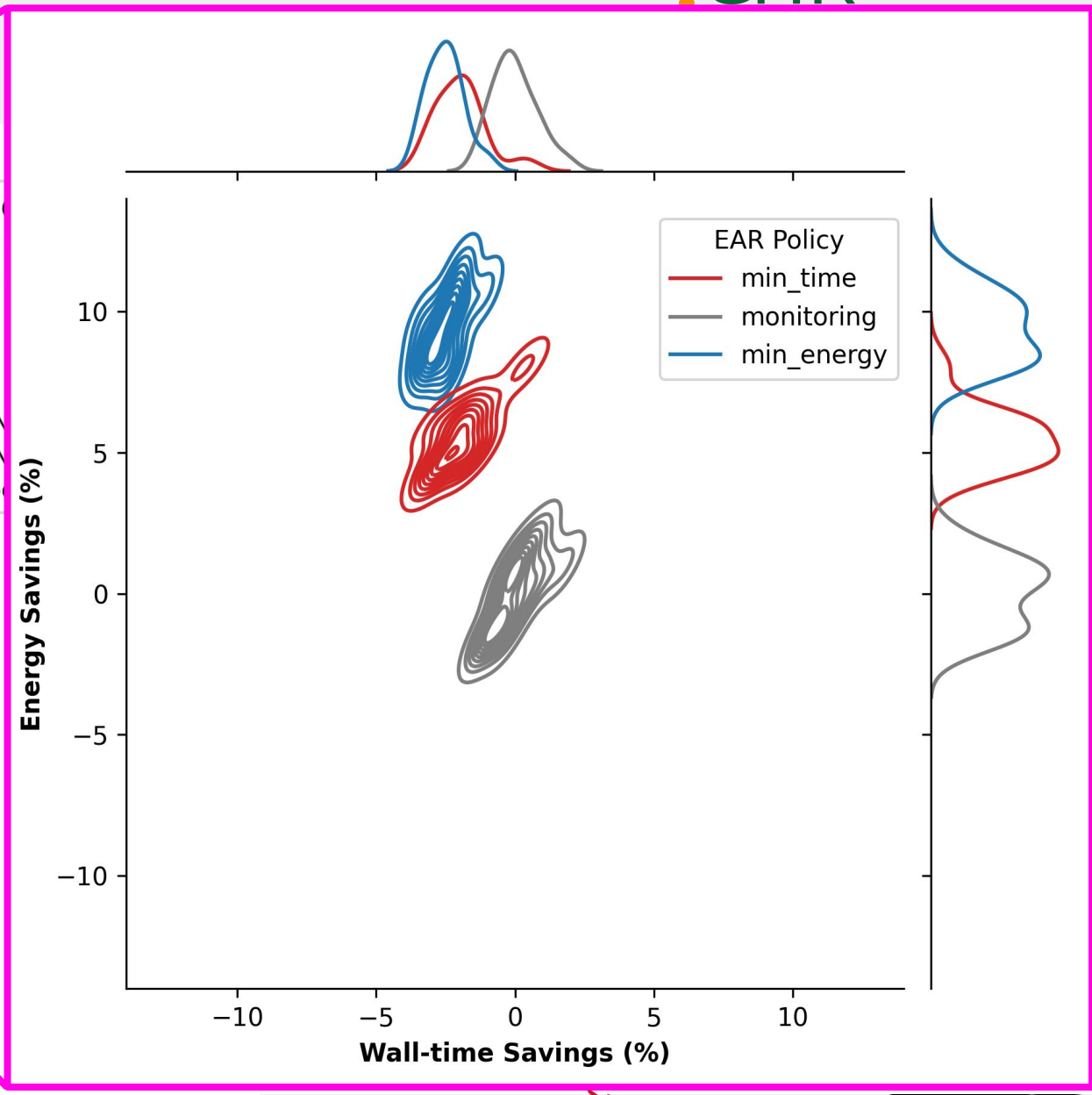
Application characterization/optimization





Application characterization/optimization





CPU advance use cases

- CPU apps
  - Palabos: monitoring and dynamic optimization
  - NPB: dynamic optimization

optimization

- Codes in /projects/0/energy-course
- GIT: https://github.com/sara-nl/ISC-EAR-tutorial/
- Get the examples, add EAR **monitoring and min_energy** policy and compare
  - –ear=on → monitoring
  - –ear-policy=min_energy → selects min_energy policy
  - Run 2 steps in the same job to guarantee both runs are executed in the same node (s)
  - https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/tutorials/policies
  - NPB : Rome vs Genoa
  - Palabos (use 1 and 4 nodes, node input_1_node_XL.xml input file)
    - Rome vs Genoa
  -

And GPU use cases

- GROMACS with SINGULARITY
  - https://gitlab.bsc.es/ear_team/ear/-/wikis/User-guide#using-earl-inside-singularity-containers
  - https://github.com/sara-nl/ISC-EAR-tutorial/blob/main/scripts/GROMACS_SINGULARITY_GPU.sh
- Extra GPU metrics. FLOPS, NVLINK, PCI…
  - https://gitlab.bsc.es/ear_team/ear/-/wikis/EAR-environment-variables#extended-gpu-metrics
- GPU optimization –ear-policy=optimize
- EAR_NTASK_WORK_SHARING
  - Ask EAR to join non-mpi applications by jobID stepID

GROMACS-GPU: Singularity + EAR

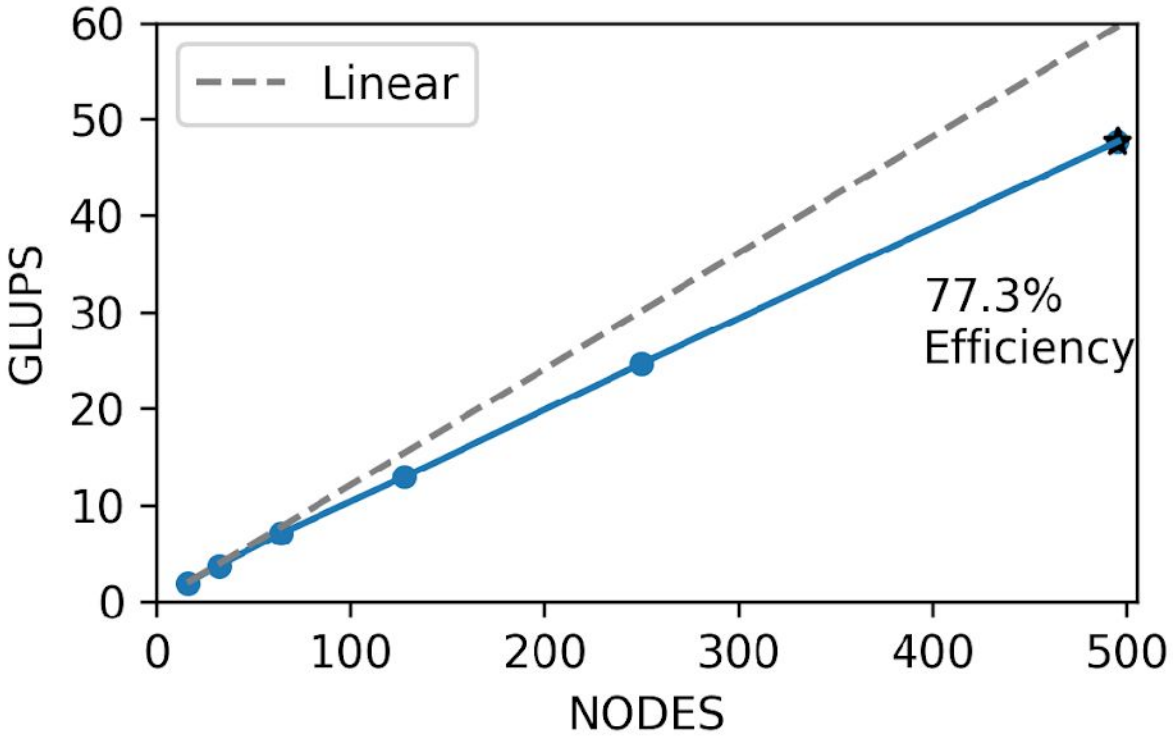
- Singularity/Apptainer
  - https://apptainer.org/
  - https://docs.sylabs.io/guides/3.5/user-guide/introduction.html
  - https://catalog.ngc.nvidia.com/orgs/hpc/collections/nvidia_hpc/entities
- Singularity containers allow applications to use host services (such as ear)
  - Paths must be binded
  - Environment variables must be defined

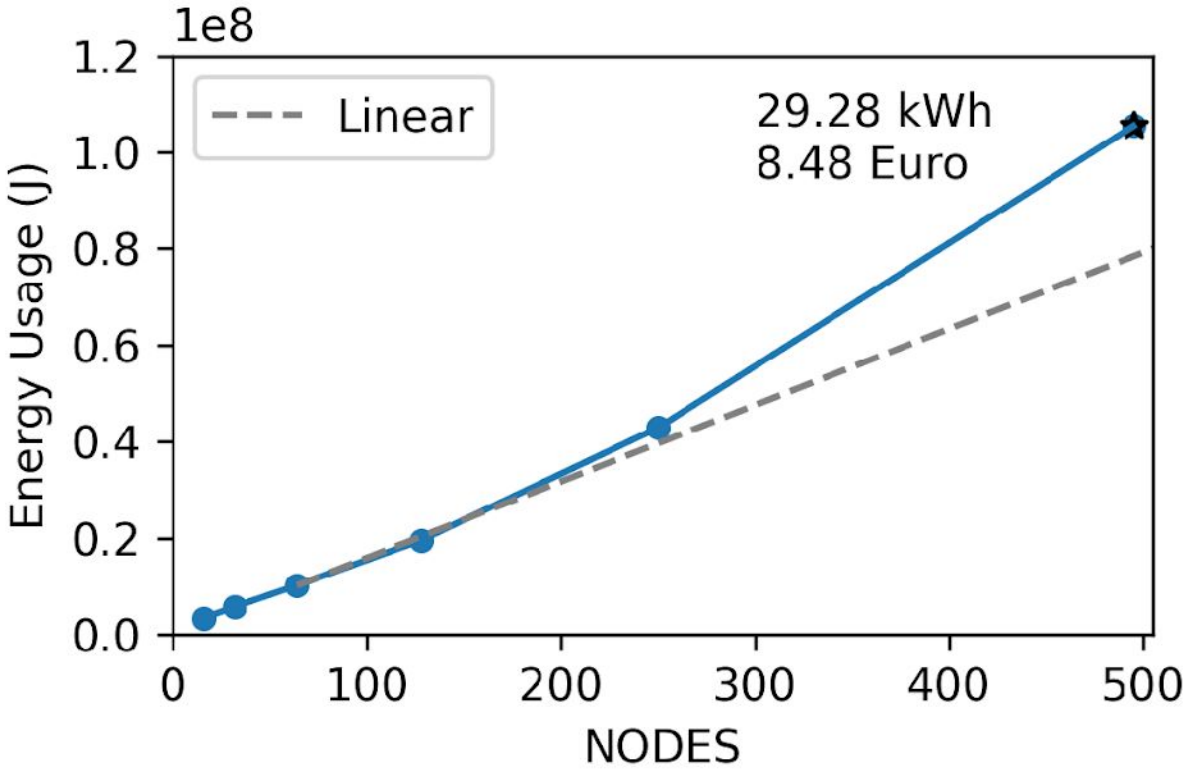
```
# BIND EAR paths
export APPTAINER_BIND="$EAR_INSTALL_PATH:$EAR_INSTALL_PATH:ro,$EAR_TMP:$EAR_TMP:rw"

# Define APPTAINER/EAR env vars
export APPTAINERENV_EAR_INSTALL_PATH=$EAR_INSTALL_PATH
export APPTAINERENV_EAR_TMP=$EAR_TMP
export APPTAINERENV_EAR_ETC=$EAR_ETC
export APPTAINERENV_EARL_REPORT_LOOPS=1
```

Palabos: Lattice-Boltzmann Solver

Strong Scaling Benchmark

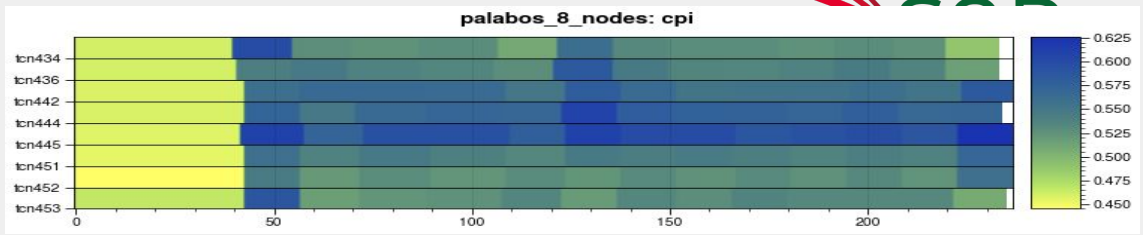


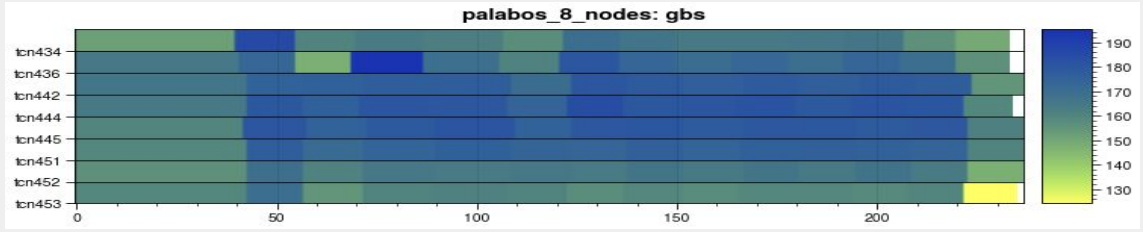


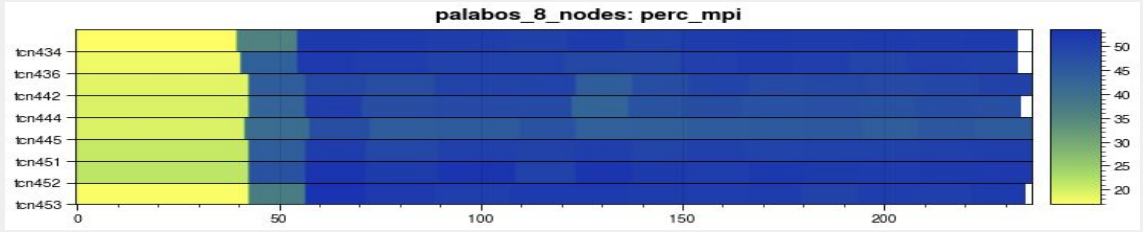
- An average electric car consumes about 0.2 kwh/km
- 495 node case (which ran for 9 minutes) just drove a car 150 km!

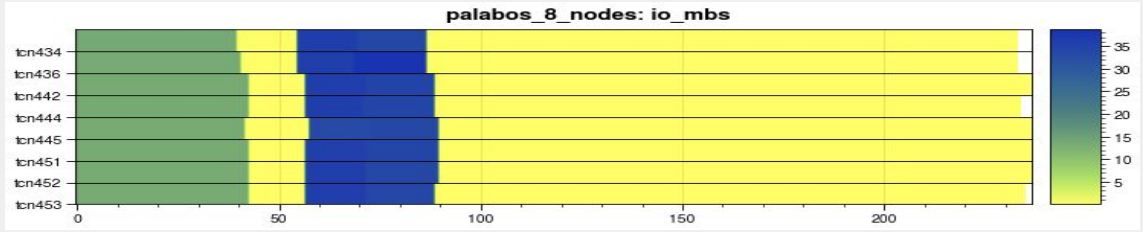
Palabos:
Strong Scaling Benchmark

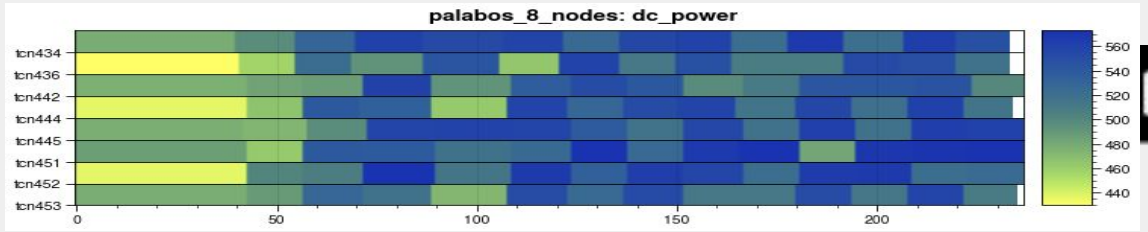
- Per-node, Per-iteration ”traces”
- Node Power
- Avg CPU Freq (node)
- Main memory BW (GB/s)
- CPI (Cycles per instruction)
- MPI% (percentage spent in MPI calls)
- I/O (network communication)



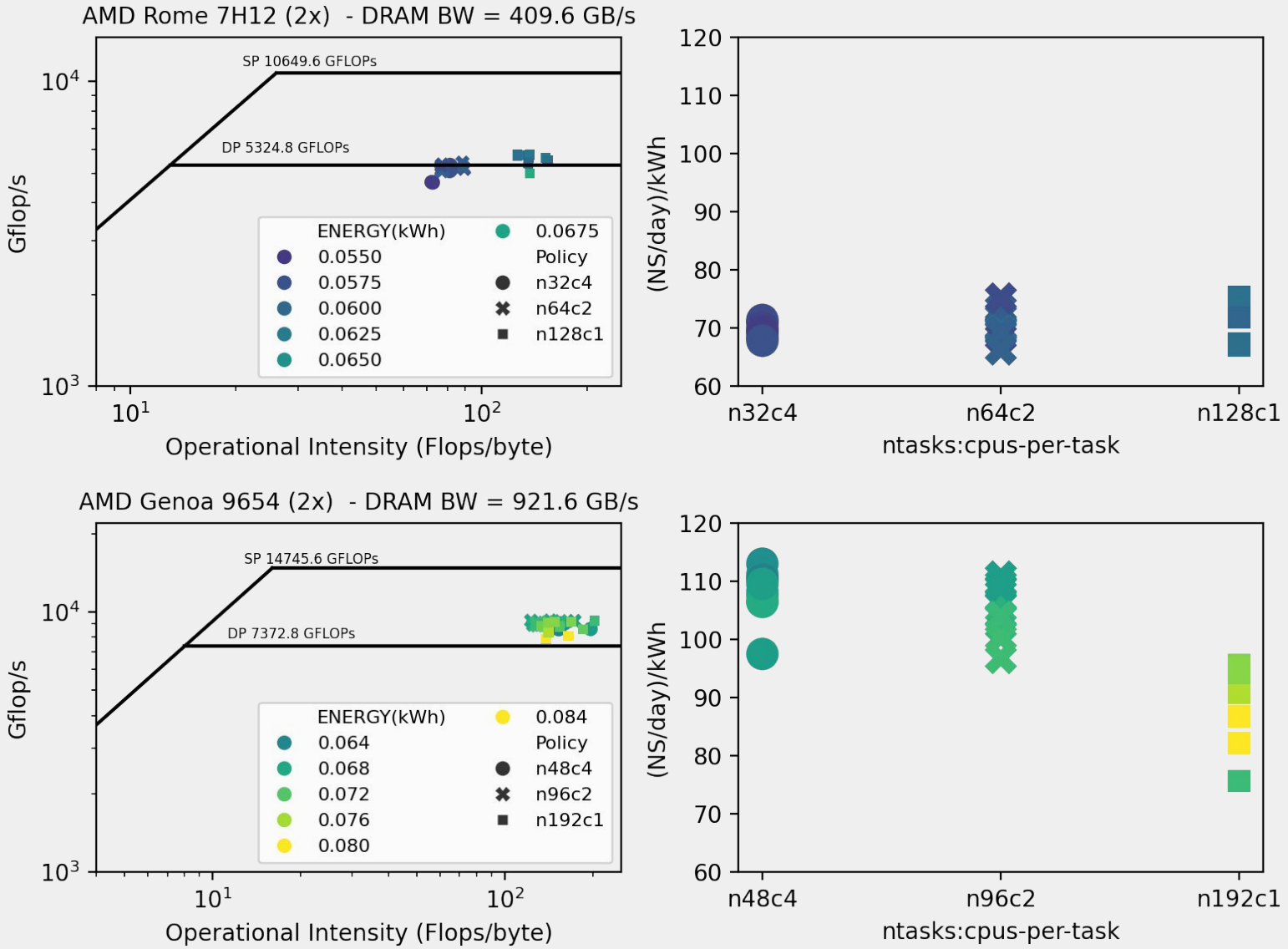




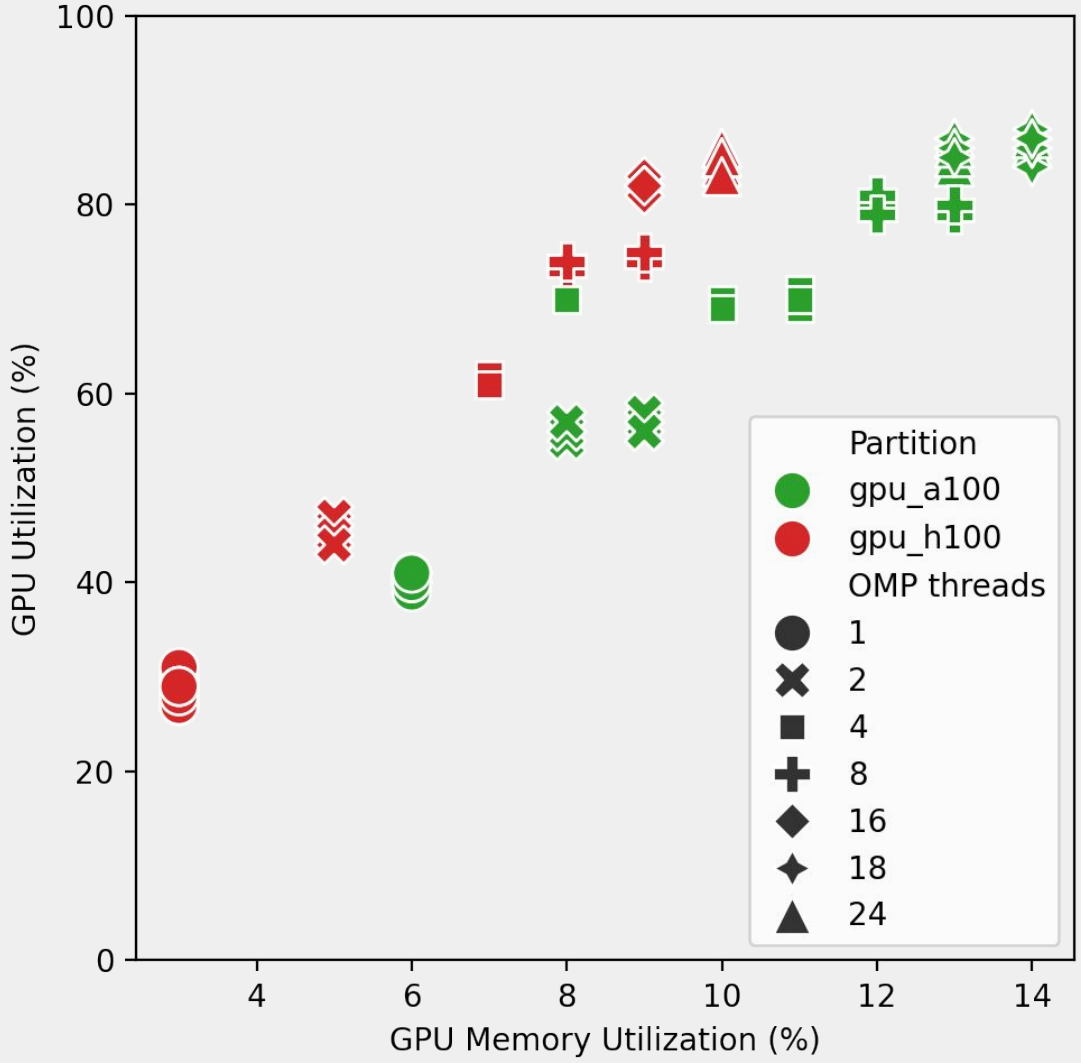


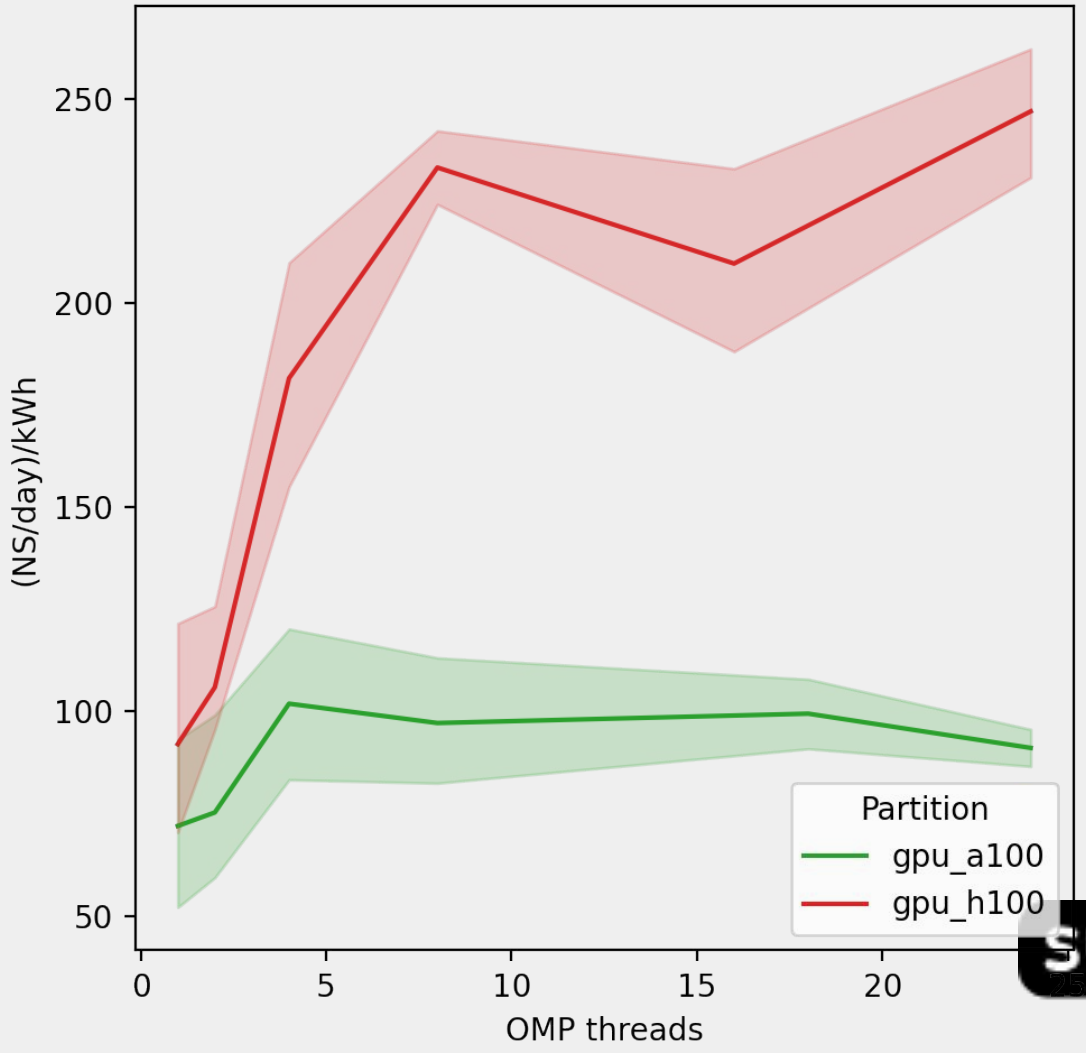


Monitoring CPU performance metrics



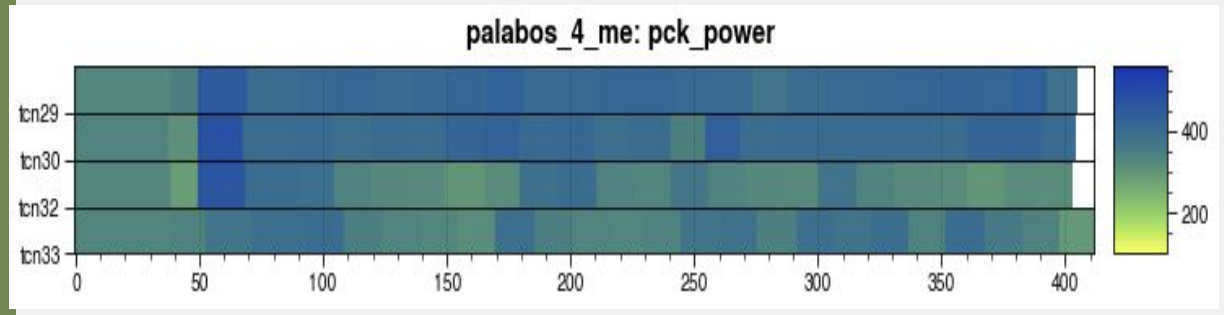
Monitoring GPU performance metrics

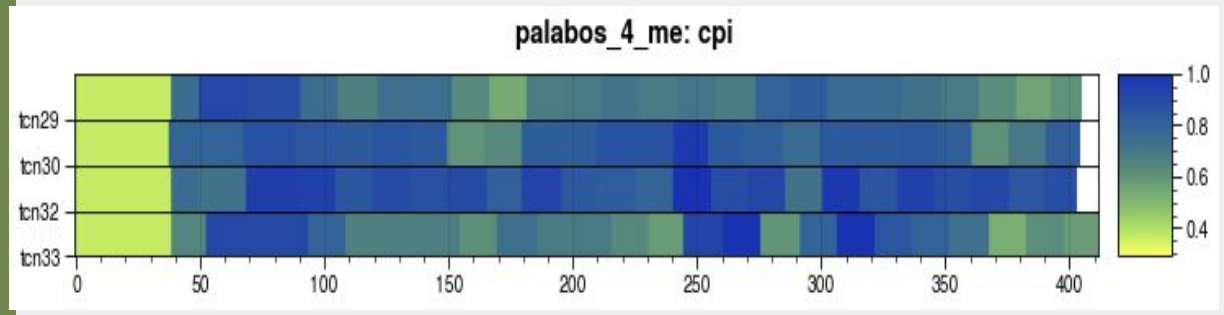


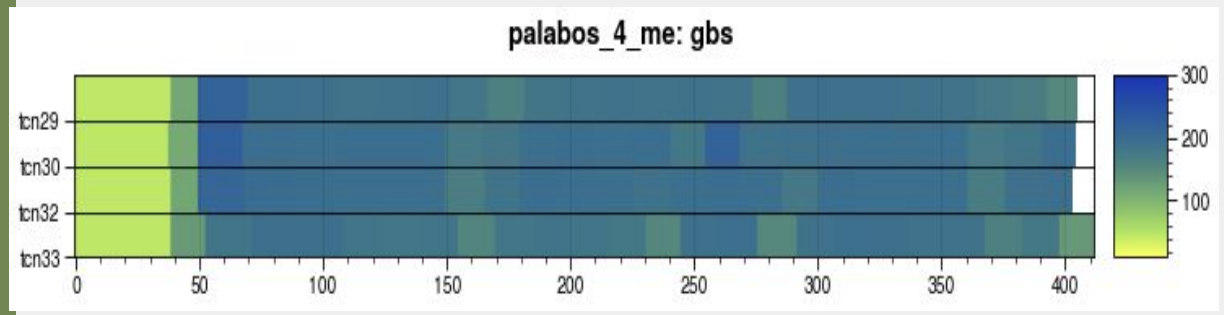


Data visualization

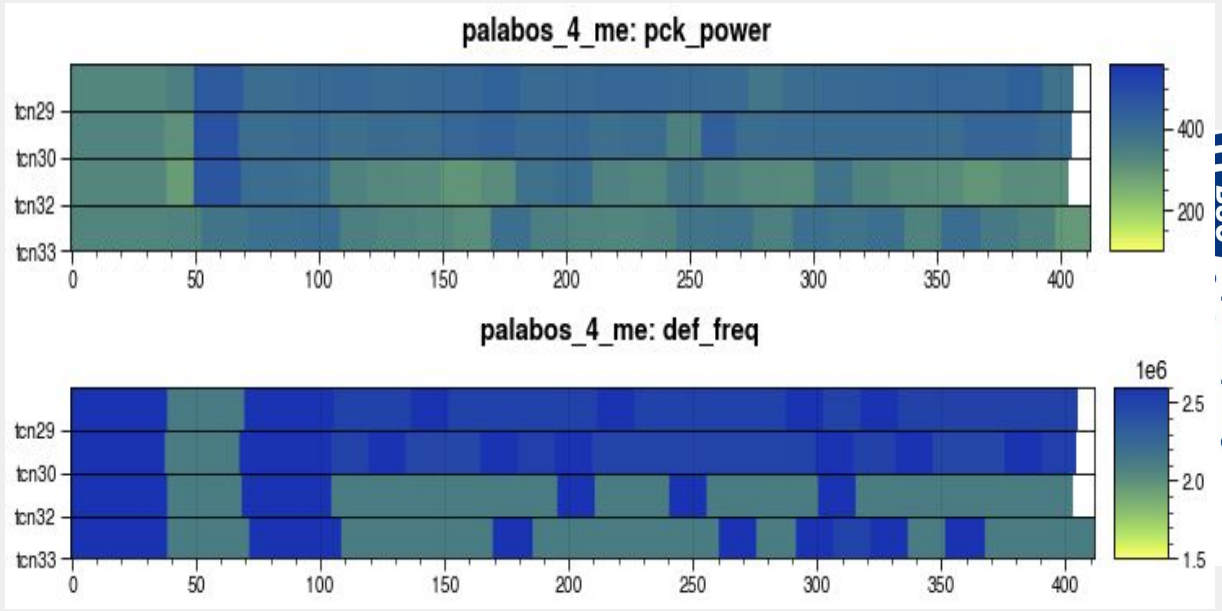
- https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/tutorials/visualization
- ear-job-visualizer
  - Requires loops in DB: export EARL_REPORT_LOOPS=1
  - Use ear-job-anaytics directly or use create_traces.sh script
- Grafana
  - Running al DC and executing SQL queries (more powerful, but depends on DC)
  - Local installation:
    - Grafana server installed and running locally
    - Data gathered in CSV format with eacct and using CSV plugin
    - Not mandatory but more information if loops are in DB
    - Can be use also without EAR DB: –ear-user-db=filename
-







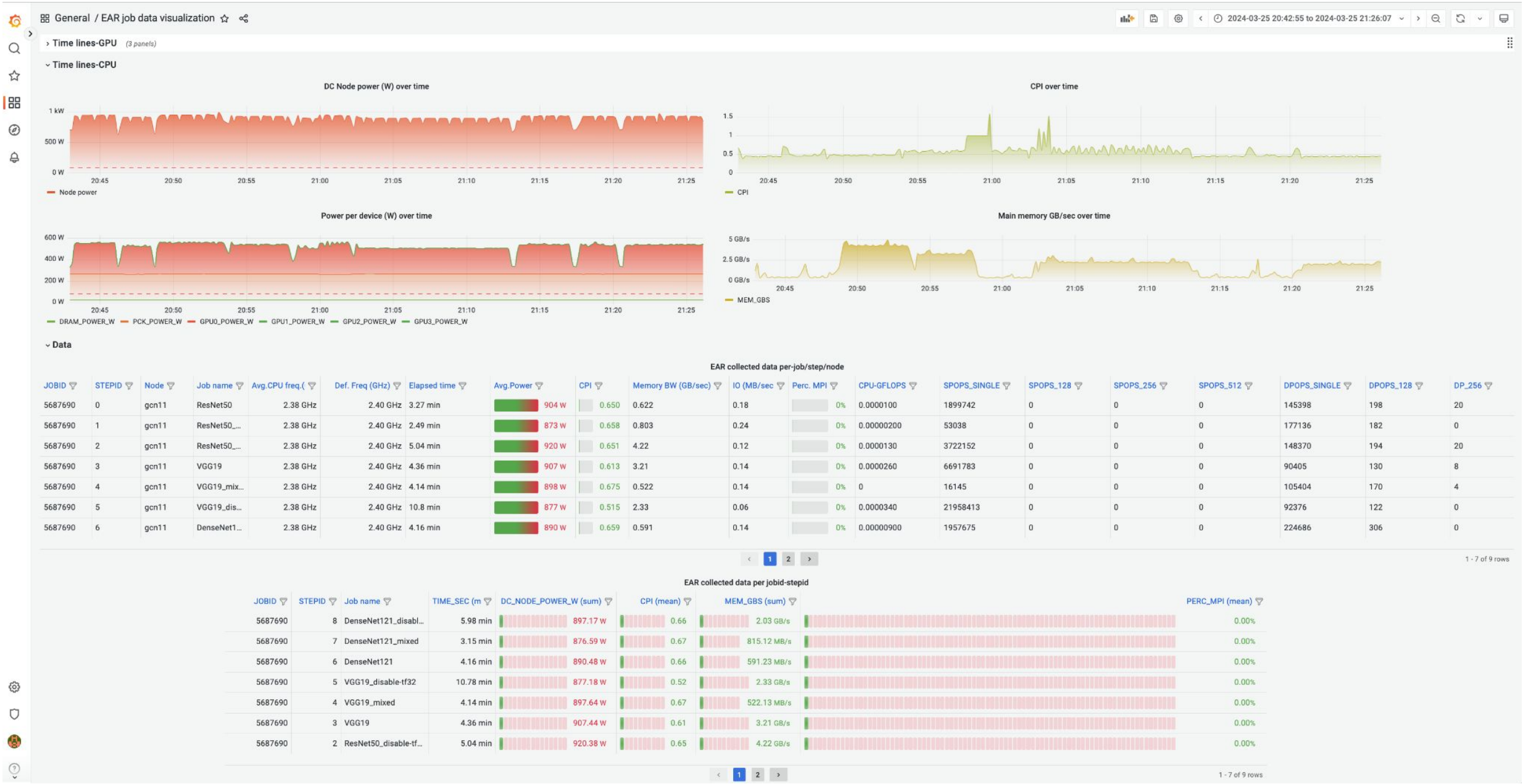
Application metrics



Runtime optimization

Visualization with Grafana

- Steps to use EAR data are in grafana with CSV are
  - Have a local grafana installation with CSV plugins supported
  - Export EAR application data in csv format using eacct (-l -c option) or –ear-user-db flag
  - Export EAR application runtime data in csv format using eacct (-r -c option) or –ear-user-db flag
  - Add a source data based on a local file (Public folder)
    - julita.corbalan$ cp tensorflow.csv ear_data_apps.csv
    - julita.corbalan$ cp tensorflow_loops.csv ear_data_loops.csv
  - Import the EAR json file with the dashboards for data visualization
    - "EAR job data visualization.json"
  - Reload the dashboards


General / EAR job data visualization
2024-03-25 20:42:55 to 2024-03-25 21:26:07
Time lines-GPU (3 panels)
Time lines-CPU
DC Node power (W) over time
1 kW
500 W
0 W
20:45 20:50 20:55 21:00 21:05 21:10 21:15 21:20 21:25
Node power
CPI over time
1.5
1
0.5
0
CPI
Power per device (W) over time
600 W
400 W
200 W
0 W
DRAM_POWER_W PCK_POWER_W GPU0_POWER_W GPU1_POWER_W GPU2_POWER_W GPU3_POWER_W
Main memory GB/sec over time
5 GB/s
2.5 GB/s
0 GB/s
MEM_GBS
Data
EAR collected data per-job/step/node
JOBID STEPID Node Job name Avg.CPU freq.(Def. Freq (GHz) Elapsed time Avg.Power CPI Memory BW (GB/sec) IO (MB/sec Perc. MPI CPU-GFLOPS SPOPS_SINGLE SPOPS_128 SPOPS_256 SPOPS_512 DPOPS_SINGLE DPOPS_128 DP_256
5687690 0 gcn11 ResNet50 2.38 GHz 2.40 GHz 3.27 min 904 W 0.650 0.622 0.18 0% 0.0000100 1899742 0 0 0 145398 198 20
5687690 1 gcn11 ResNet50_... 2.38 GHz 2.40 GHz 2.49 min 873 W 0.658 0.803 0.24 0% 0.00000200 53038 0 0 0 177136 182 0
5687690 2 gcn11 ResNet50_... 2.38 GHz 2.40 GHz 5.04 min 920 W 0.651 4.22 0.12 0% 0.0000130 3722152 0 0 0 148370 194 20
5687690 3 gcn11 VGG19 2.38 GHz 2.40 GHz 4.36 min 907 W 0.613 3.21 0.14 0% 0.0000260 6691783 0 0 0 90405 130 8
5687690 4 gcn11 VGG19_mix... 2.38 GHz 2.40 GHz 4.14 min 898 W 0.675 0.522 0.14 0% 0 16145 0 0 0 105404 170 4
5687690 5 gcn11 VGG19_dis... 2.38 GHz 2.40 GHz 10.8 min 877 W 0.515 2.33 0.06 0% 0.0000340 21958413 0 0 0 92376 122 0
5687690 6 gcn11 DenseNet1... 2.38 GHz 2.40 GHz 4.16 min 890 W 0.659 0.591 0.14 0% 0.00000900 1957675 0 0 0 224686 306 0
1 2
1 - 7 of 9 rows
EAR collected data per jobid-stepid
JOBID STEPID Job name TIME_SEC (m DC_NODE_POWER_W (sum) CPI (mean) MEM_GBS (sum) PERC_MPI (mean)
5687690 8 DenseNet121_disabl... 5.98 min 897.17 W 0.66 2.03 GB/s 0.00%
5687690 7 DenseNet121_mixed 3.15 min 876.59 W 0.67 815.12 MB/s 0.00%
5687690 6 DenseNet121 4.16 min 890.48 W 0.66 591.23 MB/s 0.00%
5687690 5 VGG19_disable-tf32 10.78 min 877.18 W 0.52 2.33 GB/s 0.00%
5687690 4 VGG19_mixed 4.14 min 897.64 W 0.67 522.13 MB/s 0.00%
5687690 3 VGG19 4.36 min 907.44 W 0.61 3.21 GB/s 0.00%
5687690 2 ResNet50_disable-tf... 5.04 min 920.38 W 0.65 4.22 GB/s 0.00%
1 2
1 - 7 of 9 rows

Go to ISC tutorial github and do the exercises

https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/tutorials/policies

HOW to…install EAR

Basic steps

1- Download last public release https://gitlab.bsc.es/ear_team/ear.git

2- Compile and deploy (use template https://github.com/sara-nl/ISC-EAR-tutorial/tree/main/scripts/ear_deploy.sh)

3- Copy $EAR_ETC/ear/ear.conf.full.template in $EAR_ETC/ear/ear.conf and adapt to your cluster

4- Create the EAR DB

5- Deploy EAR service files (available at $EAR_ETC/systemd/)

6- Enable and Start EAR service files

7- Enable EAR SLURM plugin

8- Use it !!!


CONNECTING THE DOTS
ISC
High Performance